

Understanding RAG: Retrieval-Augmented Generation

1. Introduction to RAG

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with large language model (LLM) generation. Instead of relying solely on the knowledge encoded in a model's parameters, RAG systems fetch relevant documents from an external knowledge base and use them as context for generating responses.

RAG addresses a fundamental limitation of language models: their knowledge is frozen at training time and may become outdated. By retrieving up-to-date information from a vector store or document corpus, RAG ensures that responses are grounded in current, authoritative sources.

The approach has gained significant adoption in enterprise settings where accuracy and traceability are critical. Organizations use RAG to build question-answering systems over their internal documentation, customer support knowledge bases, and technical documentation.

2. How RAG Works

2.1 Indexing Phase

The indexing phase is the foundation of any RAG system. Documents are loaded, split into chunks, and each chunk is passed through an embedding model to produce a vector representation. These vectors are stored in a vector store such as FAISS, enabling fast similarity search at query time.

2.2 Retrieval Phase

When a user submits a query, the system embeds the query using the same embedding model and searches the vector store for the most similar chunks. The retrieval step uses cosine similarity or dot product to rank candidates. Advanced systems may employ hybrid search, combining dense vector retrieval with sparse keyword search (BM25) using Reciprocal Rank Fusion (RRF) to improve recall and precision.

2.3 Generation Phase

Retrieved chunks are assembled into a prompt with the user's query and sent to a language model for generation. The model uses the provided context to produce an answer grounded in the retrieved documents, reducing hallucinations and enabling citation of sources.

3. Architecture Components

3.1 Embeddings

Embedding models convert text into dense vector representations that capture semantic meaning. Popular choices include all-MiniLM-L6-v2 for speed-optimized deployments and BAAI/bge-base-en-v1.5 for higher quality. The embedding dimension determines the vector store size and search latency. Models like jina-embeddings-v2 support longer context windows (8192 tokens) for processing extended passages.

3.2 Vector Store

The vector store holds embedding vectors and supports efficient similarity search. FAISS is a popular choice for local deployments, offering IVF and HNSW index types. For larger deployments, specialized databases like Milvus, Qdrant, or pgvector provide scalable vector storage with filtering capabilities.

3.3 Chunking

Chunking splits documents into manageable pieces for embedding and retrieval. Structure-aware chunking respects document headings and treats tables as atomic units. Common strategies include fixed-size chunks with overlap (e.g., 2000 characters, 200 overlap), sentence-based splitting, and semantic chunking. The choice of chunk size affects retrieval granularity and embedding quality.

3.4 Reranking

Reranking improves precision by reordering retrieved candidates using a cross-encoder model. After the vector store returns top-K candidates, a reranker scores each (query, chunk) pair jointly, producing a more accurate relevance score than bi-encoder embeddings alone.

4. Comparison of RAG Approaches

Aspect	Naive RAG	Advanced RAG
Indexing	Single pass, fixed chunks	Structure-aware, incremental
Retrieval	Dense vector only	Hybrid (BM25 + dense + RRF)
Embedding	Full chunk text	Preprocessed heading + summary
Reranking	None	Cross-encoder reranker
Chunking	Fixed size, no overlap	Heading-aware, table-atomic
Caching	None	LRU + TTL embedding cache
Vector Store	Single flat index	Per-corpus FAISS + backup

5. Best Practices

- Use structure-aware chunking: split on headings, keep tables atomic, and maintain overlap to preserve context across chunk boundaries.
- Preprocess chunks before embedding: send the heading plus first 500 characters to the embedding model for better semantic signal.
- Implement incremental indexing with file hashing (SHA256) to avoid re-embedding unchanged documents.
- Use hybrid search (BM25 + dense vectors) with Reciprocal Rank Fusion to capture both keyword and semantic matches.
- Cache embeddings with LRU eviction and TTL expiry to reduce compute on repeated queries. Monitor memory pressure with psutil.
- Maintain FAISS index backups (.fai.bak) before writes to enable recovery from corruption without full reindexing.
- Bind HTTP servers to localhost by default; enable authentication (Bearer token) when exposing to the network.

6. Conclusion

RAG bridges the gap between static language models and dynamic knowledge bases. By combining efficient vector retrieval with grounded generation, RAG systems deliver accurate, up-to-date answers while maintaining traceability to source documents. The architecture is flexible: embedding models can be swapped, vector stores can scale from local FAISS to distributed databases, and chunking strategies can adapt to document structure. As the ecosystem matures, techniques like hybrid search, reranking, and embedding caching continue to improve the quality and efficiency of RAG pipelines.